

DEVOPS MASTER COURSE

Volume 6F

AWS Enterprise Projects, Architecture & Interview Mastery

Chapters 56-70

Purpose Convert AWS knowledge into architecture judgment, production operating ability, evidence-backed projects, certification readiness, and senior-level interview performance.

This final part is deliberately scenario-driven. You will make decisions under constraints, implement and test systems, diagnose failure, quantify trade-offs, and produce a portfolio that demonstrates professional cloud engineering capability.

How to Use Part F

- Treat every chapter as an architecture review or production assignment, not passive reading.
- Record assumptions, requirements, alternatives, decisions, risks, evidence, and follow-up actions.
- Run labs only in controlled sandbox accounts with budgets, least privilege, and cleanup plans.
- Measure outcomes: latency, availability, RTO, RPO, deployment duration, cost, failure rate, and recovery time.
- Convert each completed project into a concise portfolio case study with diagrams and proof.

Chapter Map

Chapter	Focus	Primary outcome
56	Requirements, trade-offs, and ADRs	Defensible architecture decisions.
57	Enterprise SaaS	Tenant isolation and operability.
58	Hybrid migration	Controlled connectivity and migration waves.
59	Application modernization	Incremental monolith decomposition.
60	Multi-Region systems	Global availability and data strategy.
61	Security incident response	Containment, evidence, and recovery.
62	Resilience engineering	Game days and fault injection.
63	Performance engineering	Load models, bottlenecks, and tuning.
64	FinOps	Cost accountability and optimization.
65	Platform engineering	Golden paths and developer self-service.
66	Advanced troubleshooting	Incident command and causal diagnosis.
67	Architecture governance	Review boards, controls, and evidence.
68	Certification mastery	Exam-domain preparation and practice.
69	Interview mastery	System design and behavioral evidence.
70	Final capstone	Integrated enterprise portfolio project.

Safety and cost notice Use disposable environments, explicit budgets, limited Regions, tagged resources, and documented cleanup. Fault injection and recovery exercises must have stop conditions and must never target unmanaged production systems.

VOLUME 6F | CHAPTER 56

Chapter 56 - Architecture Requirements, Trade-offs, and ADRs

From ambiguous business goals to reviewable technical decisions

Learning Objectives

- Translate business outcomes into functional and non-functional requirements.
- Use constraints, quality attributes, risk analysis, and architectural decision records.
- Compare alternatives using measurable trade-offs rather than service preference.

Why This Matters in Production

Poor architecture usually begins before the first resource is created: unclear objectives, hidden assumptions, and unowned risks. Senior engineers make uncertainty explicit and preserve the reasoning behind consequential decisions.

Production principle Architecture is a sequence of reversible and irreversible decisions. Document context, alternatives, consequences, and validation evidence before complexity becomes institutional memory.

Working Backward from Outcomes

Start with customer and business outcomes, then derive measurable requirements. Separate mandatory constraints from preferences. Define workload scope, users, data classification, traffic shape, availability targets, recovery objectives, compliance needs, delivery timeline, team capability, and budget.

Quality attribute	Example measure	Typical design impact
Availability	99.95% monthly	Multi-AZ, health checks, automated recovery.
Latency	p95 under 250 ms	Caching, proximity, data-access design.
Durability	No acknowledged transaction loss	Replication, backups, idempotency.
Recoverability	RTO 60 min, RPO 5 min	Backup frequency, replication, runbooks.
Security	No public data plane	Private networking, identity, encryption.
Cost	Under defined unit cost	Demand model, commitments, right-sizing.

Decision Records and Lifecycle

An ADR captures title, status, context, decision, alternatives, consequences, owners, and review triggers. Accepted decisions can later be superseded without rewriting history. Link ADRs to diagrams, threat models, cost estimates, tests, and incident learnings.

- Create ADRs for structural, security, reliability, data, interface, and delivery decisions.
- Time-box investigation and use prototypes when evidence is missing.
- Identify blast radius and reversibility before approval.
- Record rejected alternatives and why they were rejected.

Trade-off Analysis

No architecture is optimal across every dimension. A managed service can reduce operations but increase dependency; multi-Region improves survivability but increases consistency and cost complexity. Use weighted decision matrices only as support - not as a substitute for engineering judgment.

Production Design Checklist

- Requirements have owners, measures, and acceptance tests.
- Assumptions and constraints are explicitly recorded.
- ADRs cover high-impact or hard-to-reverse decisions.
- Alternatives include cost, operations, security, and migration consequences.
- Architecture diagrams show trust, data, failure, and ownership boundaries.
- Review triggers exist for demand, regulation, incidents, and service changes.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Teams debate services endlessly	Outcome and constraints are undefined	Review requirements and decision criteria	Facilitate a requirements workshop and create an ADR.
Architecture changes repeatedly	Assumptions were implicit or unstable	Compare changes against original context	Version assumptions and add review triggers.
Decision cannot be explained	Reasoning exists only in meetings	Search tickets, chat, and diagrams	Write a retrospective ADR and identify evidence gaps.
Prototype looks successful but production fails	Test did not represent workload or failure modes	Compare test model with real traffic and dependencies	Repeat with representative load, security, and recovery tests.

Hands-on Lab 56: Write and Defend an Architecture Decision

Objective Produce a complete ADR for choosing the compute platform of a customer-facing API.

Implementation

1. Define workload outcomes, traffic, availability, recovery, security, team, and budget constraints.
2. Compare EC2 Auto Scaling, ECS/Fargate, EKS, and Lambda against agreed criteria.
3. Create a weighted matrix and identify where the scoring is uncertain.
4. Build a small prototype for the highest-risk assumption.
5. Write the ADR with decision, consequences, migration path, and review triggers.
6. Present the decision as a ten-minute architecture review and capture objections.

Validation Evidence

- ADR includes context, alternatives, consequences, owner, and status.
- Prototype evidence supports at least one critical assumption.
- Review comments and unresolved risks are recorded.

Cleanup

- Delete prototype resources and preserve only non-sensitive evidence.
- Archive the ADR in version control.

Stretch challenge Revisit the decision after changing one constraint, such as data residency or 10x traffic, and write a superseding ADR.

Chapter Summary

- Requirements and quality attributes make architecture reviewable.
- ADRs preserve context and consequences, not just the selected service.
- Trade-offs must be validated with measurable evidence.

Interview Questions and Model Answers

1. What makes an architecture decision significant?

It materially affects structure, security, reliability, data, interfaces, cost, or future change and is expensive or risky to reverse.

2. Why record rejected alternatives?

They prevent repeated debate and show that the selected decision was made against explicit constraints.

3. When should an ADR be reviewed?

When assumptions, constraints, service capabilities, incidents, risk, or business outcomes materially change.

Official AWS References

AWS ADR process: <https://docs.aws.amazon.com/prescriptive-guidance/latest/architectural-decision-records/adr-process.html>

ADR best practices: <https://docs.aws.amazon.com/prescriptive-guidance/latest/architectural-decision-records/best-practices.html>

Well-Architected Framework: <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>

VOLUME 6F | CHAPTER 57

Chapter 57 - Enterprise SaaS Architecture

Tenant isolation, identity, metering, onboarding, and noisy-neighbor control

Learning Objectives

- Compare pooled, siloed, and bridge tenant-isolation models.
- Design tenant-aware identity, authorization, data partitioning, observability, and metering.
- Build onboarding, offboarding, deployment, and support processes for multi-tenant systems.

Why This Matters in Production

SaaS architecture is not simply a shared web application. Every control plane and data plane must carry tenant context, enforce isolation, expose tenant-level health and cost, and support lifecycle automation.

Production principle Tenant identity must be established once, propagated explicitly, and enforced at every resource and data-access boundary. Never rely on a user-supplied tenant identifier alone.

Isolation Models

Model	Advantages	Risks / cost
Silo	Dedicated stack or account per tenant	Strong isolation; expensive and operationally heavy.
Pool	Shared resources with logical isolation	Efficient; requires rigorous tenant-aware authorization.
Bridge	Mix of shared and dedicated components	Flexible; increases operational variants and testing.

Tenant Context and Data

Use an identity provider and trusted claims to establish tenant context. Authorizers or application middleware convert identity into an internal tenant principal. Partition keys, row-level controls, schemas, databases, buckets, queues, caches, and logs must preserve that context. Test for cross-tenant access as a first-class security requirement.

Operations and Economics

Onboarding should provision configuration, entitlements, isolation tier, quotas, keys, monitoring, and billing metadata through automation. Emit tenant-aware metrics for latency, errors, throttling, consumption, and cost allocation. Control noisy neighbors with quotas, partitioning, concurrency limits, and workload isolation.

Production Design Checklist

- Tenant context originates from trusted identity and cannot be overridden by clients.
- Data access is denied by default without tenant scope.
- Cross-tenant tests run in CI and security validation.
- Onboarding and offboarding are automated and auditable.
- Per-tenant SLOs, usage, and cost attribution are available.
- Isolation tier changes have controlled migration procedures.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
One tenant sees another tenant data	Tenant filter missing or context spoofed	Trace request identity through query and logs	Contain access, rotate credentials if needed, fix enforcement, test all paths.
Large tenant degrades others	Shared quota or hot partition	Compare tenant-level traffic and throttling	Apply quotas, repartition, cache, or move tenant to isolated tier.
Support cannot diagnose tenant issue	Logs and metrics lack tenant dimensions	Inspect telemetry schema	Add bounded tenant identifiers and dashboards without exposing sensitive data.
Offboarded tenant still consumes resources	Lifecycle cleanup incomplete	Inventory resources and data by tenant tags/keys	Run deletion workflow, legal retention checks, and evidence capture.

Hands-on Lab 57: Build a Tenant-Aware Serverless API

Objective Implement a small multi-tenant API with trusted tenant claims and partitioned DynamoDB access.

Implementation

1. Create two test tenants and identities in a sandbox identity provider.
2. Build an API authorizer or trusted middleware that derives tenant context.
3. Store records with a composite key that includes tenant identity.
4. Implement create/read operations that never accept authoritative tenant context from the request body.
5. Add per-tenant metrics, throttling, and audit fields.
6. Run automated negative tests attempting cross-tenant reads and writes.

Validation Evidence

- Tenant A cannot access Tenant B records under any tested path.
- Logs and metrics identify tenant behavior without exposing secrets.
- Throttling or quotas can be demonstrated independently.

Cleanup

- Delete test identities, data, API resources, and log groups after evidence capture.

Stretch challenge Add a premium tenant tier using dedicated capacity while keeping the same application contract.

Chapter Summary

- SaaS isolation is an end-to-end property, not a database feature alone.
- Pooled, siloed, and bridge models trade efficiency against isolation and complexity.
- Tenant-aware operations and economics are required for sustainable SaaS.

Interview Questions and Model Answers

1. What is a noisy neighbor?

A tenant whose usage consumes disproportionate shared capacity and degrades other tenants.

2. How do you prevent tenant spoofing?

Derive tenant context from trusted authentication and authorization claims, then ignore or validate client-supplied tenant values.

3. When should a tenant be siloed?

When regulation, performance, contractual isolation, customization, or blast-radius requirements justify dedicated resources.

Official AWS References

AWS SaaS Lens: <https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/saas-lens.html>

SaaS Lens definitions: <https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/definitions.html>

Cognito documentation: <https://docs.aws.amazon.com/cognito/>

VOLUME 6F | CHAPTER 58

Chapter 58 - Hybrid Connectivity and Migration Waves

Discovery, landing zones, Direct Connect, VPN, DNS, identity, and cutover control

Learning Objectives

- Create an assess-mobilize-migrate plan with application dependencies and wave governance.
- Design resilient hybrid connectivity, routing, DNS, identity, and security inspection.
- Execute cutovers with replication, testing, rollback, and business communication.

Why This Matters in Production

Enterprise migrations fail when teams move servers without moving dependencies, operations, identity, or ownership. Hybrid architecture must support coexistence for months or years while remaining observable and secure.

Production principle Migrate applications as dependency-aware products, not as isolated servers. Every wave needs acceptance criteria, rollback, owner readiness, and measured business impact.

Discovery and Portfolio Planning

Inventory applications, servers, databases, interfaces, data flows, owners, criticality, compliance, lifecycle, and cost. Group systems by dependency and business process. Select migration strategies such as rehost, replatform, refactor, repurchase, retain, relocate, or retire.

Hybrid Network Design

Component	Purpose	Production concern
Site-to-Site VPN	Encrypted connectivity over internet	Bandwidth, tunnel failover, route propagation.
Direct Connect	Dedicated private connectivity	Redundant locations/connections and encryption strategy.
Transit Gateway	Hub routing across networks/accounts	Route-table segmentation and blast radius.
Route 53 Resolver	Hybrid DNS forwarding	Rule ownership, resolver endpoints, split-horizon behavior.
Network Firewall / inspection	Central traffic inspection	Symmetric routing, capacity, fail-open/closed decision.

Wave and Cutover Execution

Use replication and test environments to prove functionality, performance, security, observability, backup, and support. Freeze only what is necessary. Define the authoritative data source during transition. Use canary users or weighted DNS where possible, and preserve rollback until data divergence makes it unsafe.

Production Design Checklist

- Application dependencies are discovered and validated by owners.
- Landing zone, identity, logging, security, network, and budgets exist before migration.
- Connectivity has redundant paths and tested route failure.
- Hybrid DNS has documented authority and troubleshooting flow.
- Each wave has test, cutover, rollback, and communication plans.
- Decommission includes licenses, backups, monitoring, DNS, and security records.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Cloud workload cannot reach on-premises service	Route, security, DNS, MTU, or asymmetric path	Trace DNS, route tables, BGP, flow logs, and packet size	Correct path and retest from both directions.
Cutover produces stale data	Replication lag or dual-write ambiguity	Check replication checkpoints and writer ownership	Pause cutover, restore single writer, reconcile and retry.
VPN failover causes outage	Routes or customer gateway behavior not tested	Review tunnel/BGP state and propagation	Correct preference and run controlled failover test.
Migration wave overruns window	Dependencies or validation underestimated	Review task durations and blockers	Rollback safely, split wave, automate tests, update estimates.

Hands-on Lab 58: Plan a Three-Tier Migration Wave

Objective Create a complete migration runbook for an application with web, application, database, and on-premises dependencies.

Implementation

1. Document application inventory, owners, dependencies, RTO, RPO, and business blackout periods.
2. Choose migration strategies for each component and record ADRs.
3. Design hybrid routing, DNS, identity, logging, and security paths.
4. Create pre-cutover, cutover, validation, rollback, and decommission checklists.
5. Run a tabletop exercise with application, network, security, database, and service-desk roles.
6. Record timing, decision points, unanswered questions, and go/no-go criteria.

Validation Evidence

- Runbook identifies authoritative data and rollback boundary.
- Every validation has an owner and objective pass condition.
- Tabletop produces tracked actions and revised timing.

Cleanup

- Remove any temporary test connectivity or credentials.

Stretch challenge Create a wave scorecard that predicts migration risk and determines the required rehearsal level.

Chapter Summary

- Migration planning is dependency, ownership, and operating-model work.
- Hybrid connectivity and DNS need redundancy and evidence-based failover tests.
- Cutover must define data authority, acceptance, rollback, and communication.

Interview Questions and Model Answers

1. What is the biggest risk in rehosting?

Moving infrastructure without validating application dependencies, security, operations, performance, and licensing.

2. Why use migration waves?

They group dependency-aware workloads into manageable, repeatable cutovers and allow learning between waves.

3. When is rollback no longer safe?

When new writes or external state have diverged beyond a tested reconciliation boundary.

Official AWS References

AWS Cloud Adoption Framework: <https://docs.aws.amazon.com/whitepapers/latest/overview-aws-cloud-adoption-framework/welcome.html>

AWS Migration Hub: <https://docs.aws.amazon.com/migrationhub/>

Transit Gateway: <https://docs.aws.amazon.com/vpc/latest/tgw/>

Route 53 Resolver: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html>

VOLUME 6F | CHAPTER 59

Chapter 59 - Application Modernization and Monolith Decomposition

Strangler patterns, domain boundaries, data migration, and coexistence

Learning Objectives

- Assess whether to retain, replatform, modularize, or decompose a monolith.
- Apply strangler fig, branch by abstraction, anti-corruption layer, and event-driven integration patterns.
- Manage data ownership, consistency, deployment, observability, and organizational change.

Why This Matters in Production

Microservices are not an objective. Modernization should improve a measurable constraint such as delivery speed, scalability, resilience, maintainability, or technology lifecycle. Poor decomposition creates distributed complexity without business value.

Production principle Decompose around stable business capabilities and team ownership. Migrate incrementally with reversible coexistence, measurable outcomes, and explicit data authority.

Modernization Assessment

Measure current deployment frequency, change failure rate, recovery time, lead time, scaling constraints, defect concentration, coupling, ownership, and technology risk. A well-structured modular monolith can be better than premature microservices.

Incremental Patterns

Pattern	Use	Primary risk
Strangler fig	Intercept edge traffic and replace capabilities gradually	Routing and dual-system complexity.
Branch by abstraction	Replace deeply embedded implementation behind an interface	Temporary abstraction overhead.
Anti-corruption layer	Translate legacy and new domain semantics	Layer can become permanent technical debt.
Event-carried state transfer	Replicate needed state into service boundary	Consistency, ordering, and replay.
Transactional outbox	Publish events reliably with database change	Outbox processing and idempotency.

Data and Organizational Boundaries

A service should own its data and contract. Shared databases preserve coupling and independent deployment risk. Use CDC, events, APIs, reconciliation, and phased cutovers to move ownership. Align teams to capabilities and give them operational responsibility, not only code ownership.

Production Design Checklist

- Modernization objectives are measured and prioritized.
- Service boundaries reflect domain and ownership, not technical layers.
- Every extraction has coexistence, rollback, and decommission plans.
- Data ownership and consistency model are explicit.
- Contracts are versioned and consumers are observable.
- The new service has deployment, security, SLO, runbook, and cost ownership.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
New service causes more incidents	Boundary or operational maturity is weak	Review coupling, dependencies, SLOs, and ownership	Reduce scope, improve platform support, or merge boundary.
Legacy and new data disagree	Dual-write or replication failure	Compare event/outbox checkpoints and reconciliation	Choose source of truth, repair data, and make processing idempotent.
Migration never completes	Temporary layer lacks exit criteria or ownership	Review backlog and traffic still using legacy	Set decommission milestones and executive owner.
Chatty service calls increase latency	Boundary too fine or synchronous coupling	Trace call graph and p95 contribution	Aggregate capability, cache, or use asynchronous workflow.

Hands-on Lab 59: Extract One Capability Using the Strangler Pattern

Objective Design and prototype an incremental extraction of a notification or profile capability.

Implementation

1. Select a bounded capability with clear value and limited coupling.
2. Map current callers, data, transactions, and failure behavior.
3. Introduce a routing or abstraction seam without changing customer behavior.
4. Build the new service with its own data, identity, telemetry, and deployment.
5. Mirror or canary traffic and compare results before moving authority.
6. Document rollback, reconciliation, and legacy decommission criteria.

Validation Evidence

- Old and new paths produce equivalent approved outcomes.
- Traffic can be shifted and reversed within the defined boundary.
- The extracted service has independent SLO and ownership.

Cleanup

- Remove prototype data and infrastructure, or retain only if converted into a governed project.

Stretch challenge Add an outbox and replay process that proves recovery after an event consumer outage.

Chapter Summary

- Modernization must target measurable constraints, not fashion.
- Incremental coexistence patterns reduce transformation risk.
- Data authority and team ownership determine whether a service is truly independent.

Interview Questions and Model Answers

1. What is the strangler fig pattern?

An incremental modernization approach in which new capabilities are built around and gradually replace parts of a legacy system.

2. Why are shared databases risky for microservices?

They couple schemas, releases, permissions, performance, and ownership, reducing independent change.

3. When should you keep a monolith?

When the domain, team scale, operational maturity, and change patterns do not justify distributed-system cost.

Official AWS References

Decomposing monoliths: <https://docs.aws.amazon.com/prescriptive-guidance/latest/modernization-decomposing-monoliths/introduction.html>

Strangler fig pattern: <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/strangler-fig.html>

Anti-corruption layer: <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/acl.html>

VOLUME 6F | CHAPTER 60

Chapter 60 - Multi-Region and Global Application Design

Latency, sovereignty, failover, data replication, and operational independence

Learning Objectives

- Choose active-passive, active-active, or partitioned global architectures.
- Design DNS, traffic, identity, data, secrets, observability, and deployment across Regions.
- Define failover authority, consistency, data residency, and recovery testing.

Why This Matters in Production

Multi-Region architecture can reduce regional dependency but adds complex failure modes. Global systems must survive not only infrastructure loss but control-plane isolation, stale data, partial replication, operator confusion, and unsafe failback.

Production principle Use multi-Region only when business objectives justify its ongoing cost and complexity. Each Region should be as operationally independent as the recovery objective requires.

Global Architecture Patterns

Pattern	Best fit	Trade-off
Backup and restore	Longer RTO/RPO workloads	Lowest steady-state cost, slow recovery.
Pilot light	Core data/services always available	Moderate recovery time and operating complexity.
Warm standby	Scaled-down full environment	Faster recovery, continuous duplication cost.
Active-active	Very low RTO, global demand	Highest data and operations complexity.
Cell / geography partition	Fault isolation by customer or geography	Routing and capacity planning complexity.

Traffic and Data

Route 53, Global Accelerator, and CloudFront can steer traffic, but health criteria must represent user success and dependency health. Data strategy may use Aurora Global Database, DynamoDB global tables, S3 replication, event replication, or application-level conflict handling. Define acceptable consistency and conflict resolution per data domain.

Failover and Failback

Pre-provision quotas, keys, secrets, certificates, dependencies, and observability. Automate safe steps but preserve an accountable incident commander for irreversible data decisions. Failback is a separate migration: reconcile data, restore capacity, validate, shift gradually, and prevent split-brain.

Production Design Checklist

- Business-approved regional failure objectives exist.
- Regional dependencies and hidden global control planes are identified.
- Data consistency, conflict resolution, and authoritative writer rules are documented.
- Traffic health checks test business transactions, not instance reachability alone.
- Capacity and quotas support failover demand.
- Failover and fallback are exercised with measured evidence.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Traffic fails over but app is broken	Health check is too shallow or dependency unavailable	Run synthetic transaction and inspect regional dependencies	Improve health criteria and replicate missing dependencies.
Two Regions accept conflicting writes	Writer fencing or conflict model absent	Inspect timestamps, versions, and routing history	Stop dual writes, select authority, reconcile, implement conflict strategy.
Failover Region cannot scale	Quotas, reservations, or warm capacity missing	Check service quotas and capacity errors	Pre-raise quotas and maintain tested standby capacity.
Fallback causes data loss	Reconciliation and freeze boundary incomplete	Compare replication lag and divergent records	Pause, reconcile, validate, and shift traffic gradually.

Hands-on Lab 60: Run a Regional Failure Tabletop

Objective Design and rehearse failure of the primary Region for a transactional application.

Implementation

1. Document architecture, global dependencies, data stores, health checks, RTO, and RPO.
2. Define detection, declaration, communication, traffic shift, writer fencing, and validation steps.
3. Inject a simulated regional dependency failure in a nonproduction environment.
4. Measure detection, decision, recovery, and data-lag times.
5. Rehearse fallback as a separate controlled migration.
6. Produce a post-game report and prioritized remediation backlog.

Validation Evidence

- Measured recovery is compared with RTO/RPO.
- Data-authority decisions and stop conditions are explicit.
- Remediation actions have owners and due dates.

Cleanup

- Return traffic and data to normal state and delete temporary test resources.

Stretch challenge Automate one reversible failover step and prove it stops when validation fails.

Chapter Summary

- Multi-Region architecture trades regional resilience for system complexity.
- Traffic, data, identity, security, observability, and operations all require regional design.
- Failback needs the same rigor as failover.

Interview Questions and Model Answers

1. Active-active or active-passive: which is better?

Neither universally; choose based on RTO, RPO, consistency, traffic, operational capability, and cost.

2. Why can DNS failover be slow or unsafe?

Caching, resolver behavior, shallow health checks, and stale sessions can delay or misdirect traffic.

3. What is split-brain?

Multiple locations independently accepting conflicting authority for the same state.

Official AWS References

AWS Disaster Recovery guidance: <https://docs.aws.amazon.com/whitepapers/latest/disaster-recovery-workloads-on-aws/disaster-recovery-options-in-the-cloud.html>

Route 53 failover: <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-failover.html>

Global Accelerator: <https://docs.aws.amazon.com/global-accelerator/>

DynamoDB global tables: <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GlobalTables.html>

VOLUME 6F | CHAPTER 61

Chapter 61 - Security Incident Response and Cloud Forensics

Preparation, detection, containment, evidence preservation, eradication, and recovery

Learning Objectives

- Build a cloud incident-response plan with roles, access, tooling, and evidence paths.
- Investigate identity, network, compute, storage, and data events using AWS telemetry.
- Contain incidents through reversible automation while preserving evidence and business continuity.

Why This Matters in Production

Cloud incident response is fastest when evidence, access, roles, and containment actions are prepared before an alert. Improvised response can destroy evidence, spread compromise, or create a larger outage.

Production principle Preserve evidence and establish scope before destructive remediation. Containment must be reversible, authorized, observable, and proportional to the confirmed risk.

Preparation

Create a security incident account or isolated tooling environment, delegated security services, centralized logs, forensic roles, protected evidence buckets, tested contact paths, playbooks, and legal/compliance escalation. Use MFA and tightly governed break-glass access.

Investigation Sequence

Question	Evidence
Who acted?	CloudTrail identity, session context, source IP, MFA, role chain.
What changed?	CloudTrail events, Config timeline, deployment history.
What communicated?	VPC Flow Logs, DNS logs, load balancer, WAF, application logs.
What data was accessed?	S3 data events, database audit logs, application authorization logs.
What executed?	EDR/runtime telemetry, process and package evidence, snapshots/images.
How far did it spread?	Organization-wide findings, credential use, network and resource inventory.

Containment and Recovery

Examples include disabling or scoping a session, isolating a security group, applying an SCP to a compromised account, revoking credentials, blocking indicators, quarantining an image, or redirecting traffic. Prefer snapshots and immutable evidence before termination. Rebuild from trusted artifacts, rotate secrets, verify persistence removal, and monitor for recurrence.

Production Design Checklist

- Incident roles, severity, declaration, communication, and evidence ownership are documented.
- CloudTrail and relevant data events are centralized and protected.
- Forensic access and tooling are pre-provisioned and tested.
- Containment actions are scripted, reviewed, and reversible where possible.
- Evidence handling records time, source, integrity, and custodian.
- Recovery includes root cause, credential rotation, validation, and lessons learned.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
No CloudTrail evidence	Trail missing, Region excluded, or retention expired	Check organization trails, event history, log bucket, and service events	Restore logging, use alternate evidence, document gap and prevention.
Containment breaks production	Dependency or scope was not understood	Review change, service map, and blast radius	Rollback containment, use narrower control, improve playbook.
Attacker activity continues after rotation	Other sessions, keys, roles, tokens, or persistence remain	Enumerate credentials, role sessions, resources, and automation	Revoke all paths, rebuild compromised assets, continue monitoring.
Evidence timeline conflicts	Clock, timezone, ingestion, or duplicated events	Normalize timestamps and source semantics	Build a confidence-labeled timeline and preserve raw records.

Hands-on Lab 61: Investigate a Simulated Credential Compromise

Objective Run a controlled tabletop and log investigation for suspicious API activity in a sandbox account.

Implementation

1. Create a benign sequence of unusual API calls using a dedicated training role.
2. Collect CloudTrail, Config, GuardDuty or equivalent findings, and resource metadata.
3. Build a normalized event timeline and identify the assumed role chain.
4. Choose a reversible containment action and document approval and blast radius.
5. Preserve evidence metadata and then restore or rebuild the affected resource.
6. Write an incident report with root cause, impact, detection, response, and preventive controls.

Validation Evidence

- Timeline is reproducible from source evidence.
- Containment stops the simulated path without unnecessary account outage.
- Report clearly separates facts, inferences, and unknowns.

Cleanup

- Delete training resources and revoke the dedicated training role.
- Retain sanitized incident evidence only according to policy.

Stretch challenge Automate enrichment of a finding with account owner, resource tags, recent CloudTrail events, and recommended reversible containment.

Chapter Summary

- Cloud response requires pre-built access, evidence, roles, and playbooks.
- Identity and control-plane evidence are central to cloud forensics.
- Containment, eradication, and recovery are distinct phases with different risks.

Interview Questions and Model Answers

1. Should you terminate a compromised instance immediately?

Usually not before evaluating business risk and preserving evidence; isolation and snapshotting may be safer first actions.

2. What is chain of custody?

A record showing how evidence was collected, transferred, protected, and handled to preserve integrity and trust.

3. Why use a separate forensic role or account?

It reduces dependence on potentially compromised identities and centralizes protected investigation tooling.

Official AWS References

AWS Security Incident Response Guide: <https://docs.aws.amazon.com/whitepapers/latest/aws-security-incident-response-guide/welcome.html>

CloudTrail: <https://docs.aws.amazon.com/cloudtrail/>

AWS Config: <https://docs.aws.amazon.com/config/>

GuardDuty: <https://docs.aws.amazon.com/guardduty/>

VOLUME 6F | CHAPTER 62

Chapter 62 - Resilience Engineering, Game Days, and Fault Injection

Hypothesis-driven experiments, stop conditions, recovery measurement, and learning

Learning Objectives

- Define resilience hypotheses from failure modes and business objectives.
- Design safe game days using AWS Fault Injection Service and controlled operational experiments.
- Measure detection, containment, recovery, and organizational response.

Why This Matters in Production

Backups and redundancy create potential resilience; only testing demonstrates actual resilience. Game days expose hidden dependencies, unsafe automation, missing permissions, poor alarms, and unclear decision authority before a real incident.

Production principle Every experiment needs a hypothesis, steady-state measure, bounded blast radius, stop condition, owner, and rollback. The objective is learning - not causing disruption.

Experiment Design

Begin with a failure-mode analysis: instance loss, AZ impairment, dependency latency, DNS failure, expired certificate, throttling, queue backlog, bad deployment, or corrupted data. Define expected system behavior and measurable steady state.

Safety Controls

Control	Purpose
Target selection	Limit by account, tags, resource IDs, and environment.
Stop conditions	Terminate experiment when alarms or thresholds breach.
Duration and rate	Bound exposure and allow observation.
Approvals	Ensure workload and business owners accept risk.
Rollback / recovery	Restore state and verify data and customer outcomes.
Observer roles	Separate operators, incident commander, and recorder.

Learning and Maturity

Record actual detection time, page delivery, diagnosis, decision, containment, recovery, and customer impact. Convert findings into engineering backlog, runbook changes, alarms, tests, architecture improvements, and future experiments. Repeat experiments after remediation.

Production Design Checklist

- Failure hypotheses map to workload objectives and known risks.
- Experiments run first in representative nonproduction environments.

- Targets, duration, stop conditions, and rollback are tested.
- Observers know communication and escalation roles.
- Customer-facing steady state is measured during experiment.
- Findings become owned remediation with retest dates.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Experiment affects unintended resources	Targeting or tags too broad	Inspect FIS target resolution and resource tags	Stop experiment, restore service, fix targeting and policy.
Stop condition does not trigger	Alarm missing data, wrong dimension, or delay	Review alarm state and metric path	Create faster protective signal and test before rerun.
System recovers but data is wrong	Steady state measured availability only	Run reconciliation and integrity checks	Add correctness SLI and recovery validation.
Game day becomes blame exercise	Learning culture and roles unclear	Review facilitation and postmortem language	Focus on system conditions, evidence, and actionable controls.

Hands-on Lab 62: Run an Auto Scaling Resilience Experiment

Objective Test whether a stateless service remains healthy when instances are terminated.

Implementation

1. Define the hypothesis, steady-state request success and latency, target group, stop alarms, and rollback.
2. Run baseline load in a sandbox multi-AZ service.
3. Create an FIS experiment or controlled automation that terminates a bounded percentage of instances.
4. Observe alarms, Auto Scaling replacement, load balancer health, and customer SLI.
5. Record detection and recovery times and any capacity or deployment issues.
6. Write findings, implement one improvement, and repeat the experiment.

Validation Evidence

- The experiment stays within target scope and stop conditions.
- Measured recovery is compared with the hypothesis.
- The repeated experiment demonstrates the remediation effect.

Cleanup

- Stop load generation and delete experiment templates and sandbox resources.

Stretch challenge Extend the experiment to an AZ-scoped impairment or dependency latency scenario with a stricter stop condition.

Chapter Summary

- Resilience is demonstrated by controlled failure and recovery evidence.
- Fault experiments must be hypothesis-driven and bounded by safety controls.
- Game-day value comes from remediation and repeat testing.

Interview Questions and Model Answers

1. What is steady state?

A measurable description of acceptable customer and system behavior before, during, and after an experiment.

2. Why are stop conditions essential?

They automatically limit harm when protective metrics indicate the experiment is exceeding safe boundaries.

3. How is chaos engineering different from random failure?

It is a disciplined experiment with a hypothesis, scope, safety controls, observations, and learning outcomes.

Official AWS References

AWS Fault Injection Service: <https://docs.aws.amazon.com/fis/>

Reliability Pillar: <https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/welcome.html>

Resilience Hub: <https://docs.aws.amazon.com/resilience-hub/>

VOLUME 6F | CHAPTER 63

Chapter 63 - Performance Engineering and Capacity Planning

Demand models, load testing, observability, bottlenecks, and safe tuning

Learning Objectives

- Create realistic demand and concurrency models from business behavior.
- Design load, stress, spike, soak, and failure-under-load tests.
- Diagnose bottlenecks across client, edge, network, compute, runtime, cache, queue, and data layers.

Why This Matters in Production

Performance is an end-to-end customer property. Adding compute may hide a database, lock, dependency, or serialization bottleneck while increasing cost. Production performance requires representative tests and correlated telemetry.

Production principle Optimize the limiting resource only after reproducing the workload, measuring the full request path, and protecting correctness and reliability.

Workload Model

Model requests per second, concurrency, object sizes, read/write mix, key distribution, session behavior, geographic origin, background work, cache hit rate, and growth. Include realistic think time and warm-up. Avoid using averages to represent peaks.

Test Types

Test	Question answered
Load	Does the system meet objectives at expected demand?
Stress	Where does it fail and how does it degrade?
Spike	Can it absorb rapid demand change?
Soak	Do leaks, queues, or fragmentation appear over time?
Scalability	Does capacity add useful throughput predictably?
Failover under load	Does recovery work at real demand?

Diagnosis and Tuning

Correlate user latency with load balancer, compute, runtime, thread/connection pool, cache, queue, and database metrics plus traces and profiles. Use utilization, saturation, errors, and queueing. Tune one variable at a time, record before/after, and verify cost and failure behavior.

Production Design Checklist

- Performance objectives use percentiles and error budgets.
- Test data and access patterns represent production.

- Load generation is isolated from the system under test.
- Autoscaling min/max and cooldown behavior are validated.
- Database, cache, queue, and dependency limits are observed.
- Tuning changes have rollback and before/after evidence.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Latency rises while CPU is low	I/O, lock, connection, dependency, or queue bottleneck	Inspect traces, saturation, waits, and pool metrics	Fix limiting dependency or concurrency control.
Load test results vary widely	Warm-up, cache, data, environment, or generator differs	Compare run configuration and environment state	Standardize test harness and reset conditions.
Autoscaling reacts too late	Metric or warm-up does not lead demand	Review scaling timeline and queue/concurrency signal	Use better leading metric, predictive/scheduled capacity, or warm pool.
Higher throughput corrupts data	Race, idempotency, or transaction assumptions fail	Inspect duplicate and concurrency paths	Fix correctness before further performance tuning.

Hands-on Lab 63: Benchmark and Tune a Web API

Objective Measure a sandbox API, identify one bottleneck, tune it, and prove the improvement.

Implementation

1. Define p50, p95, p99 latency, error, throughput, and cost objectives.
2. Create representative test data and a controlled load profile.
3. Run a baseline while collecting metrics, logs, traces, and database/cache telemetry.
4. Identify the limiting resource using evidence rather than utilization guesses.
5. Apply one reversible tuning change.
6. Repeat the exact test and compare performance, reliability, and cost.

Validation Evidence

- Test configuration and environment are reproducible.
- Before/after evidence shows whether the hypothesis was correct.
- Correctness remains intact under peak load.

Cleanup

- Stop generators and remove sandbox data and capacity.

Stretch challenge Run the tuned service through failover under load and compare recovery with the normal-load game day.

Chapter Summary

- Performance requires realistic demand models and end-to-end telemetry.
- Percentiles, saturation, queues, and traces reveal bottlenecks better than average CPU.

- Every tuning change must preserve correctness, resilience, and cost efficiency.

Interview Questions and Model Answers

1. What is the difference between load and stress testing?

Load tests expected demand against objectives; stress tests exceed expected demand to identify limits and degradation behavior.

2. Why can averages be misleading?

They hide tail latency and short peaks that often determine customer experience and timeouts.

3. What is saturation?

The degree to which a resource has queued work or no remaining capacity to accept more demand.

Official AWS References

Performance Efficiency Pillar: <https://docs.aws.amazon.com/wellarchitected/latest/performance-efficiency-pillar/welcome.html>

Distributed Load Testing solution: <https://docs.aws.amazon.com/solutions/latest/distributed-load-testing-on-aws/solution-overview.html>

CloudWatch: <https://docs.aws.amazon.com/cloudwatch/>

VOLUME 6F | CHAPTER 64

Chapter 64 - FinOps and Enterprise Cost Optimization

Allocation, unit economics, commitments, anomaly response, and value engineering

Learning Objectives

- Establish cost ownership, allocation, budgets, forecasts, and anomaly response.
- Optimize rate, usage, architecture, licensing, storage, and operational waste.
- Use unit economics and engineering metrics to prioritize sustainable savings.

Why This Matters in Production

Cost optimization is not periodic deletion or indiscriminate downsizing. It is a continuous operating practice that connects product demand, architecture, reliability, and financial accountability.

Production principle Optimize cost per business outcome while protecting security, reliability, performance, and delivery speed. Every saving should have an owner, evidence, and regression monitor.

Cost Visibility and Ownership

Define account, workload, environment, owner, product, cost-center, and data-classification allocation standards. Activate cost allocation tags, use account boundaries, allocate shared costs deliberately, and reconcile invoices with product metrics.

Optimization Levers

Lever	Examples	Risk to manage
Eliminate waste	Idle resources, unattached volumes, stale snapshots	Deleting required recovery or forensic assets.
Right-size usage	Compute, database, cache, provisioned throughput	Performance and failover headroom.
Rate optimization	Savings Plans, Reserved Instances, Spot	Commitment and interruption risk.
Architecture	Serverless, caching, event-driven, storage tiers	Complexity and lock-in.
Data transfer	Topology, CDN, endpoints, compression	Latency and security path changes.
Licensing	BYOL, managed service, edition changes	Compliance and feature compatibility.

Unit Economics and Governance

Examples include cost per tenant, order, active user, build, GB processed, or million requests. A unit metric separates healthy business growth from inefficient growth. Run a recurring FinOps cadence with engineering, finance, and product: review anomalies, forecasts, commitments, unit trends, and optimization backlog.

Production Design Checklist

- Allocation coverage and unallocated spend are measured.

- Budgets and anomaly alerts route to accountable owners.
- Commitments follow stable measured usage and scenario analysis.
- Optimization preserves tested performance and recovery headroom.
- Shared costs use a documented allocation method.
- Unit cost and savings verification are part of product reviews.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Spend rises with no traffic growth	Leak, logging, transfer, or configuration change	Use Cost Explorer/CUR by service, usage type, account, and tag	Contain anomaly, fix cause, add monitor.
Savings Plan underutilized	Commitment exceeds eligible steady usage	Review utilization and coverage	Adjust future purchases and shift eligible workloads carefully.
Downsizing causes outage	Peak/failover capacity was ignored	Compare demand percentiles and recovery tests	Restore capacity, use autoscaling and safer headroom policy.
Unallocated cost is high	Tagging/account model incomplete	Report missing dimensions and shared services	Enforce allocation at provisioning and define shared-cost model.

Hands-on Lab 64: Create a Cost Optimization Business Case

Objective Analyze a sandbox or sample workload and produce a prioritized, validated savings plan.

Implementation

1. Define workload owner, business unit, demand metric, SLO, and current monthly cost.
2. Break cost down by service, account, Region, usage type, and allocation dimension.
3. Identify at least five opportunities across waste, size, rate, data transfer, storage, or architecture.
4. Estimate gross savings, implementation effort, reliability risk, and payback.
5. Implement one low-risk change and validate performance and cost.
6. Create a monthly unit-cost dashboard and regression alert.

Validation Evidence

- Business case includes assumptions and confidence level.
- Implemented change has measured before/after evidence.
- Unit cost is linked to a workload demand metric.

Cleanup

- Delete temporary analysis resources; retain sanitized reports.

Stretch challenge Build a commitment scenario comparing on-demand, Savings Plans, and seasonal demand with failover headroom.

Chapter Summary

- FinOps connects cloud spend to ownership and business value.
- Optimization includes usage, rate, architecture, transfer, storage, and licensing.
- Unit economics and regression monitoring make savings durable.

Interview Questions and Model Answers

1. What is the difference between coverage and utilization?

Coverage is the share of eligible usage receiving a discounted rate; utilization is how much of the purchased commitment is consumed.

2. Why is unit cost important?

It distinguishes cost growth caused by valuable demand from cost growth caused by inefficiency.

3. Should every idle resource be deleted?

No; validate ownership, recovery, compliance, forensic, and scheduled-use requirements first.

Official AWS References

Cost Optimization Pillar: <https://docs.aws.amazon.com/wellarchitected/latest/cost-optimization-pillar/welcome.html>

AWS Cost Management: <https://docs.aws.amazon.com/cost-management/>

Cost and Usage Reports: <https://docs.aws.amazon.com/cur/>

VOLUME 6F | CHAPTER 65

Chapter 65 - Platform Engineering and Internal Developer Platforms

Golden paths, self-service, product thinking, policy, and platform SLOs

Learning Objectives

- Define platform customers, products, interfaces, and measurable adoption outcomes.
- Create golden paths that combine infrastructure, delivery, observability, security, and operations.
- Operate a platform with versioning, support, SLOs, feedback, and paved-road governance.

Why This Matters in Production

Cloud expertise does not scale if every application team assembles security, networking, pipelines, observability, and operations independently. Platform engineering converts repeated organizational knowledge into safe, supported products.

Production principle A golden path should make the secure, reliable, observable way the easiest way while preserving an explicit, governed escape hatch for exceptional workloads.

Platform as a Product

Identify developer personas and pain through research. Define platform capabilities such as account vending, service templates, CI/CD, secrets, observability, runtime, data access, and cost visibility. Track adoption, time to first deploy, lead time, failure rate, support load, and developer satisfaction.

Golden Path Architecture

Layer	Platform capability
Portal / API	Service catalog, ownership, documentation, self-service actions.
Templates	Approved application and infrastructure starters.
Delivery	Standard build, test, security, promotion, and rollback.
Runtime	Supported compute patterns and scaling defaults.
Observability	Metrics, logs, traces, dashboards, SLOs, alerts.
Governance	Policy, identity, network, tagging, cost, exception workflow.

Versioning and Escape Hatches

Treat templates and constructs as versioned APIs. Publish deprecations and migration paths. Provide constrained extension points. Exceptions require business justification, risk owner, compensating controls, expiry, and plan to return to supportable patterns.

Production Design Checklist

- Platform capabilities map to measured developer problems.
- Golden paths include security, reliability, observability, cost, and operations.

- Templates and constructs are tested, versioned, documented, and supported.
- Platform SLOs and ownership are explicit.
- Exceptions are visible, time-bound, and reviewed.
- Feedback and adoption data drive the roadmap.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Teams bypass platform	Path is slow, restrictive, or poorly documented	Interview teams and analyze time-to-delivery	Fix product gap, simplify interface, or create governed extension.
Platform change breaks many teams	Versioning or compatibility is weak	Map consumers and release history	Restore prior version, add contract tests and migration plan.
Self-service creates insecure resources	Policy and template controls incomplete	Inspect generated IaC and identity	Add preventive checks, safe defaults, and approval for risky actions.
Platform team becomes ticket queue	Automation and product boundaries unclear	Categorize requests and recurring work	Automate high-volume paths and define supported interfaces.

Hands-on Lab 65: Build a Golden Path Service Template

Objective Create a reusable service starter that includes deployment, security, observability, and operations.

Implementation

1. Choose one workload archetype such as a stateless API or event consumer.
2. Create application and IaC templates with required ownership and environment metadata.
3. Add pipeline stages for tests, scanning, deployment, health validation, and rollback.
4. Provision standard logs, metrics, tracing, dashboard, alarms, and runbook link.
5. Document supported configuration, extension points, and exception process.
6. Test onboarding with another person and measure time to first successful deployment.

Validation Evidence

- A new service deploys without manual cloud-console configuration.
- Security and observability controls are present by default.
- Onboarding time and user feedback are recorded.

Cleanup

- Destroy the sandbox service while preserving reusable template source.

Stretch challenge Publish the template as a catalog item with version pinning and an automated upgrade report.

Chapter Summary

- Platform engineering packages organizational knowledge into supported products.
- Golden paths reduce cognitive load and variance while preserving controlled exceptions.

- Adoption, reliability, and developer outcomes measure platform value.

Interview Questions and Model Answers

1. What is a golden path?

A supported, opinionated route for creating and operating a common workload with safe defaults and integrated controls.

2. How is a platform different from a collection of scripts?

A platform has customers, interfaces, ownership, reliability, documentation, versioning, support, and a product roadmap.

3. Why provide escape hatches?

Not all workloads fit one pattern; governed exceptions prevent the platform from blocking legitimate requirements or driving shadow infrastructure.

Official AWS References

AWS Prescriptive Guidance - platform engineering: <https://docs.aws.amazon.com/prescriptive-guidance/latest/platform-engineering/welcome.html>

AWS Proton: <https://docs.aws.amazon.com/proton/>

AWS Service Catalog: <https://docs.aws.amazon.com/servicecatalog/>

VOLUME 6F | CHAPTER 66

Chapter 66 - Advanced Troubleshooting and Incident Command

Evidence-first diagnosis, distributed failure, mitigation, and learning

Learning Objectives

- Use a repeatable troubleshooting method across identity, network, compute, data, and distributed systems.
- Run incident command with clear roles, communication, decisions, and mitigation priorities.
- Build causal timelines and blameless post-incident improvements.

Why This Matters in Production

During an outage, unstructured parallel changes create more uncertainty. Senior operators stabilize customer impact, preserve evidence, coordinate specialists, and distinguish correlation from cause.

Production principle Mitigate customer impact first with the smallest reversible action, while maintaining a shared timeline and testing one hypothesis at a time.

Troubleshooting Method

Confirm the symptom and customer scope. Establish the last known good state and change timeline. Verify account, Region, identity, and environment. Trace one failed request end to end. Check dependencies in order. Form a falsifiable hypothesis, run the smallest diagnostic, and record the outcome.

Incident Command

Role	Responsibility
Incident commander	Owens priorities, decisions, safety, and coordination.
Operations lead	Executes mitigation and technical work.
Communications lead	Publishes stakeholder and customer updates.
Scribe	Maintains timeline, decisions, hypotheses, and actions.
Subject expert	Investigates a bounded domain under command direction.

Distributed Failure Patterns

Watch for retry storms, cascading timeouts, queue growth, partial deployment, DNS caching, clock skew, stale credentials, hot partitions, connection-pool exhaustion, exhausted ephemeral ports, quota failures, and observability gaps. A local error can be downstream impact, not root cause.

Production Design Checklist

- Severity and incident declaration criteria are documented.
- Roles and communication channels are assigned immediately.
- All changes and observations enter a shared timestamped timeline.
- Mitigation actions are reversible and validated against customer signals.
- Hypotheses are explicitly confirmed or rejected.

- Post-incident actions address detection, prevention, containment, and recovery.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Many services fail simultaneously	Shared dependency, identity, DNS, network, or deployment	Find common path and first failure in timeline	Mitigate shared dependency and stop unrelated changes.
Metrics look healthy but users fail	Wrong dimensions or telemetry gap	Run synthetic request and inspect client/edge/data path	Add customer-centric SLI and missing instrumentation.
Rollback does not restore service	Data/schema/config or dependency changed separately	Compare full release set and irreversible state	Restore compatible configuration/data or forward-fix safely.
Incident has repeated status confusion	No command structure or source of truth	Review channels, timeline, and decision ownership	Assign roles and publish fixed-cadence updates.

Hands-on Lab 66: Run an Incident Simulation

Objective Facilitate a timed outage exercise for a failing API with a database dependency and recent deployment.

Implementation

1. Prepare scenario injects, expected evidence, safety boundaries, and observers.
2. Declare the incident and assign commander, operations, communications, and scribe roles.
3. Provide incomplete evidence gradually and require explicit hypotheses.
4. Require a customer-impact mitigation before root-cause remediation.
5. Produce stakeholder updates at a fixed cadence.
6. Complete a post-incident review with system-focused actions and owners.

Validation Evidence

- Timeline records decisions, evidence, and rejected hypotheses.
- Mitigation reduces the defined customer impact metric.
- Postmortem actions are specific, owned, and testable.

Cleanup

- Reset the training environment and remove scenario credentials/data.

Stretch challenge Rerun the simulation after remediation and compare detection, mitigation, and recovery times.

Chapter Summary

- Evidence-first troubleshooting reduces random change and confirmation bias.
- Incident command creates clarity under pressure.
- Post-incident learning must improve the system, not assign blame.

Interview Questions and Model Answers

1. What is the first priority during a severe incident?

Protect people and reduce customer/business impact while maintaining safe coordination and evidence.

2. What makes a good hypothesis?

It is specific, consistent with evidence, and can be confirmed or rejected through a bounded test.

3. Why separate mitigation from root-cause repair?

A fast reversible mitigation can restore service before the deeper and riskier corrective change is understood.

Official AWS References

Operational Excellence Pillar: <https://docs.aws.amazon.com/wellarchitected/latest/operational-excellence-pillar/welcome.html>

CloudWatch: <https://docs.aws.amazon.com/cloudwatch/>

AWS Systems Manager Incident Manager: <https://docs.aws.amazon.com/incident-manager/>

VOLUME 6F | CHAPTER 67

Chapter 67 - Architecture Review Boards, Compliance, and Evidence

Risk-based governance without delivery paralysis

Learning Objectives

- Design lightweight architecture and security review stages based on risk.
- Automate preventive and detective controls while preserving accountable exceptions.
- Produce auditable evidence from source, pipeline, cloud configuration, and operations.

Why This Matters in Production

Governance fails when it is either absent or purely bureaucratic. Effective architecture review concentrates expert attention on irreversible, high-blast-radius, regulated, or unfamiliar decisions while automating routine controls.

Production principle Govern decisions and risk, not diagrams for their own sake. Evidence should be generated continuously by delivery and operations rather than reconstructed before an audit.

Risk-Tiered Review

Tier	Example	Review approach
Standard	Approved golden path with no sensitive data	Automated policy and asynchronous review.
Elevated	New data store, internet exposure, cross-account access	Domain expert review and threat model.
Critical	Regulated data, multi-Region authority, new identity model	Formal board, security, legal/compliance, game-day evidence.

Evidence Architecture

Evidence sources include version control approvals, ADRs, threat models, IaC tests, artifact provenance, pipeline records, CloudTrail, Config, security findings, patch compliance, backups, restore tests, incident records, cost allocation, and exception registers. Centralize evidence with immutable retention and access separation.

Exception Management

An exception must identify the violated requirement, business need, risk, compensating control, owner, approval, scope, monitoring, expiry, and remediation plan. Expired exceptions must fail closed into review rather than silently persist.

Production Design Checklist

- Review tier is based on objective risk factors.
- Standard patterns receive fast automated approval.
- Critical decisions have ADR, threat model, recovery, and operations evidence.
- Controls are mapped to technical evidence and owners.
- Exceptions are time-bound, monitored, and visible.
- Review outcomes and recurring issues improve the platform roadmap.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Review takes weeks	All changes follow the same manual process	Analyze risk tiers and queue time	Automate standard path and reserve board for high-risk decisions.
Audit evidence is missing	Controls do not emit or retain proof	Map each control to source and retention	Integrate evidence generation into pipeline and operations.
Exception becomes permanent	No expiry, owner, or tracking	Review exception register and alerts	Set expiry, escalate, remediate, or formally accept residual risk.
Teams hide changes from governance	Process adds delay without value	Gather feedback and compare bypass causes	Improve service level, transparency, and paved-road capabilities.

Hands-on Lab 67: Run a Risk-Tiered Architecture Review

Objective Review three proposed changes and produce decisions, evidence requirements, and conditions.

Implementation

1. Define review tiers and scoring factors for data, exposure, privilege, blast radius, novelty, and recoverability.
2. Evaluate a standard web service, a cross-account data pipeline, and a regulated multi-Region workload.
3. Request only evidence proportional to each risk tier.
4. Record decisions, conditions, unresolved risks, and review triggers.
5. Create one automated control for the standard path.
6. Create an exception record for one intentionally unmet control.

Validation Evidence

- Each change receives a consistent risk tier and rationale.
- Decision records identify evidence, owner, and expiry conditions.
- Automated control reduces manual review for the standard pattern.

Cleanup

- Remove training exceptions and test resources.

Stretch challenge Create an evidence manifest that links one deployed workload to every required control and its source artifact.

Chapter Summary

- Architecture governance should accelerate safe standard work and focus experts on material risk.
- Continuous evidence is more reliable than audit-time reconstruction.
- Exceptions require ownership, compensating controls, monitoring, and expiry.

Interview Questions and Model Answers

1. What should an architecture review board review?

High-impact decisions, risks, alternatives, and evidence - not every low-risk implementation detail.

2. What is continuous compliance?

Automated, ongoing evaluation and evidence generation integrated into delivery and operations.

3. Why are time-bound exceptions important?

They prevent temporary risk acceptance from becoming invisible permanent architecture.

Official AWS References

AWS Audit Manager: <https://docs.aws.amazon.com/audit-manager/>

AWS Config: <https://docs.aws.amazon.com/config/>

AWS Artifact: <https://docs.aws.amazon.com/artifact/>

Well-Architected Tool: <https://docs.aws.amazon.com/wellarchitected/latest/userguide/intro.html>

VOLUME 6F | CHAPTER 68

Chapter 68 - AWS Certification Mastery

Domain mapping, scenario reasoning, labs, and exam readiness

Learning Objectives

- Build a certification plan aligned to job role and hands-on gaps.
- Use official exam guides to map domains to evidence-backed practice.
- Apply a structured method to complex multiple-choice and multiple-response scenarios.

Why This Matters in Production

Certification can validate broad knowledge, but memorizing service facts is not enough for professional-level exams or real work. The strongest preparation combines official scope, hands-on implementation, troubleshooting, architecture trade-offs, and timed scenario practice.

Production principle Use the current official exam guide as the source of truth. Treat every weak domain as a practical lab and architecture decision, not a flashcard list.

Choosing a Path

Goal	Common certification direction
Cloud architecture foundation	Solutions Architect - Associate, followed by Professional when experience supports it.
Operations / cloud operations	CloudOps Engineer - Associate, then DevOps Engineer - Professional.
Development	Developer - Associate, then DevOps Engineer - Professional.
Security specialization	Security - Specialty after strong identity, logging, network, data, and incident skills.
Data / machine learning	Role-aligned associate or specialty credentials based on current AWS catalog.

Exam Preparation System

Download the current official exam guide and build a domain matrix: concept confidence, hands-on evidence, troubleshooting evidence, and practice accuracy. Use spaced review, labs, architecture whiteboarding, and timed question sets. Review wrong answers by explaining why every distractor is wrong under the scenario constraints.

Scenario Question Method

Identify the primary requirement, hard constraints, and optimization priority. Eliminate answers that violate security, availability, operations, cost, or migration constraints. Prefer managed, automated, scalable solutions when they satisfy the exact requirement - but do not choose a service merely because it is managed.

Production Design Checklist

- Official current exam guide is downloaded and version recorded.
- Study plan maps every domain to labs and evidence.
- Weak topics are tested through implementation and troubleshooting.
- Practice review explains both correct and incorrect choices.
- Timed mock performance is stable before booking.
- Exam preparation does not replace production safety or real experience.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Practice scores plateau	Review is passive or errors are not classified	Categorize misses by knowledge, reading, trade-off, or time	Create targeted labs and teach-back notes.
Many questions feel ambiguous	Primary requirement and constraints are missed	Annotate scenario and compare choices	Use requirement-first elimination.
Service facts are forgotten	No retrieval practice or implementation	Map facts to architecture and CLI/console lab	Use spaced recall and practical repetition.
Candidate rushes final questions	Timing strategy untested	Review time per question and flagged items	Run timed mocks and use a fixed flag-and-return process.

Hands-on Lab 68: Build a 30-Day Certification Sprint

Objective Create and execute a role-aligned certification study plan based on an official exam guide.

Implementation

1. Select one certification and download the current official exam guide.
2. Create a domain matrix with confidence and evidence columns.
3. Schedule daily review, three weekly labs, two architecture sessions, and timed practice.
4. Complete one troubleshooting lab for each major domain.
5. Run two timed mock exams and analyze every error.
6. Write a readiness decision with remaining risks and next actions.

Validation Evidence

- Plan is traceable to the official exam domains.
- Every weak domain has practical evidence.
- Mock performance and time management improve across attempts.

Cleanup

- Delete lab resources and store only sanitized notes.

Stretch challenge Create twenty original scenario questions from your project evidence and peer-review the distractors.

Chapter Summary

- Certification preparation should follow current official scope.
- Hands-on and troubleshooting evidence convert memorization into durable skill.
- Scenario questions are solved by requirements and trade-offs, not keyword matching.

Interview Questions and Model Answers

1. How should you use practice exams?

As diagnostic tools to identify reasoning and knowledge gaps, not as question banks to memorize.

2. What is the best certification preparation?

A combination of relevant hands-on experience, official exam scope, labs, architecture reasoning, and timed practice.

3. How do you handle two plausible answers?

Identify the scenario priority and choose the option that satisfies all constraints with the least unnecessary operational burden or risk.

Official AWS References

AWS Certification exam guides: <https://docs.aws.amazon.com/aws-certification/latest/examguides/aws-certification-exam-guides.html>

DevOps Engineer Professional guide: <https://docs.aws.amazon.com/aws-certification/latest/devops-engineer-professional-02/devops-engineer-professional-02.html>

Solutions Architect Professional guide: <https://docs.aws.amazon.com/aws-certification/latest/solutions-architect-professional-02/solutions-architect-professional-02.html>

AWS Certification preparation: <https://aws.amazon.com/certification/certification-prep/>

VOLUME 6F | CHAPTER 69

Chapter 69 - AWS Technical Interview and System Design Mastery

Structured communication, deep troubleshooting, trade-offs, and behavioral evidence

Learning Objectives

- Answer architecture and troubleshooting questions with a repeatable structure.
- Demonstrate depth in identity, networking, reliability, data, delivery, security, and cost.
- Convert project experience into concise behavioral evidence using measurable outcomes.

Why This Matters in Production

Senior interviews evaluate how you reason under ambiguity, communicate trade-offs, detect risk, and learn from failure. A list of AWS services does not demonstrate architecture judgment or operational ownership.

Production principle Clarify requirements before designing. State assumptions, make trade-offs explicit, test failure modes, and connect every technical choice to a measurable outcome.

System Design Structure

Clarify users, traffic, data, consistency, availability, latency, RTO/RPO, security, compliance, team, timeline, and budget. Present a simple baseline architecture first. Then deepen data, failure, security, operations, deployment, and cost. Identify bottlenecks and evolution path.

Troubleshooting Interview

Use symptom -> scope -> last known good -> changes -> request path -> evidence -> hypothesis -> test -> mitigation -> prevention. Ask for metrics and logs rather than guessing. Explain which action is reversible and how customer impact is validated.

Behavioral Evidence

Element	What strong evidence contains
Situation	Business and technical context, scale, and stakes.
Task	Your explicit responsibility and constraints.
Action	Decisions, collaboration, technical work, and trade-offs.
Result	Measured outcome, customer effect, cost, reliability, or delivery change.
Learning	What failed, what changed, and how prevention was verified.

Production Design Checklist

- Answers begin with requirements and constraints.
- Architecture includes identity, security, observability, deployment, recovery, and cost.
- Trade-offs and rejected alternatives are explained.
- Troubleshooting uses evidence and reversible mitigation.

- Behavioral stories quantify outcomes and personal contribution.
- Unknowns are acknowledged with a plan to validate.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Answer becomes a service list	No requirements or architecture flow	Pause and restate primary constraints	Draw end-to-end request/data flow and justify each component.
Candidate overclaims experience	Personal contribution and evidence unclear	Ask what they directly decided or implemented	Use precise scope and honest learning.
Design ignores failure	Only happy path discussed	Ask what fails first and how detected	Add failure modes, recovery, and data consequences.
Candidate gets stuck	Question is broad or unfamiliar	Clarify constraints and decompose	State assumptions and reason from first principles.

Hands-on Lab 69: Run a Senior Cloud Interview Panel

Objective Complete a mock system-design, troubleshooting, and behavioral interview with evidence-based feedback.

Implementation

1. Prepare a one-page architecture and experience portfolio.
2. Answer a 45-minute system-design prompt without starting with services.
3. Complete a 20-minute outage diagnosis using staged evidence.
4. Answer three behavioral questions using measurable STAR-L stories.
5. Score clarity, requirements, trade-offs, security, reliability, operations, cost, and ownership.
6. Repeat the weakest section after focused practice.

Validation Evidence

- Second attempt improves measured score.
- Answers distinguish facts, assumptions, and validation plans.
- Behavioral stories include personal actions and outcomes.

Cleanup

- Remove any confidential employer information from recordings or notes.

Stretch challenge Create a question bank of fifty prompts and map each to one portfolio artifact that proves competence.

Chapter Summary

- Strong interviews demonstrate reasoning, not service memorization.
- System design should progress from requirements to baseline, risks, and evolution.
- Operational and behavioral evidence distinguish experienced engineers.

Interview Questions and Model Answers

1. How do you start a system-design interview?

Clarify users, scale, data, quality attributes, constraints, and success criteria before selecting technology.

2. What do you do when you do not know a service feature?

State the uncertainty, reason from requirements, propose a safe alternative, and explain how you would verify the detail.

3. What makes a behavioral story senior-level?

Clear ownership, cross-team judgment, difficult trade-offs, measurable impact, and learning that changed the system.

Official AWS References

AWS Architecture Center: <https://aws.amazon.com/architecture/>

Well-Architected Framework: <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>

AWS Certification exam guides: <https://docs.aws.amazon.com/aws-certification/latest/examguides/aws-certification-exam-guides.html>

VOLUME 6F | CHAPTER 70

Chapter 70 - Final Integrated Enterprise Capstone and Portfolio

Design, build, secure, operate, test, optimize, and defend a production-grade AWS platform

Learning Objectives

- Integrate the full Volume 6 skill set into a governed enterprise workload.
- Produce implementation, security, reliability, operational, cost, and recovery evidence.
- Present the architecture and incident decisions to a senior review panel.

Why This Matters in Production

The final capstone is the bridge from learning to professional evidence. It requires more than a functioning application: the platform must be deployable, observable, secure, recoverable, cost-accountable, and explainable.

Production principle No capstone claim is accepted without evidence. Every requirement must map to an implementation, test, measurement, owner, and residual-risk statement.

Business Scenario

Build a multi-tenant commerce or case-management platform used in two geographies. It requires web and API access, asynchronous order processing, transactional data, analytics export, least-privilege operations, controlled delivery, tenant-level observability, regional recovery, and cost allocation.

Required Architecture

Domain	Minimum evidence
Foundation	Multi-account or logically separated environments, identity, budgets, audit logging.
Network	Multi-AZ VPC, private workloads, controlled ingress/egress, DNS.
Compute	Container or serverless workload with autoscaling and immutable release.
Data	Transactional store, backups, encryption, recovery test, data classification.
Integration	Queue/event/workflow with idempotency, DLQ, and replay.
Delivery	IaC, tests, scanning, artifact integrity, staged promotion, rollback.
Operations	Metrics, logs, traces, SLOs, alarms, dashboards, runbooks.
Security	Threat model, secrets, findings, incident playbook, evidence.
Resilience	Failure experiment and regional recovery tabletop or test.
FinOps	Allocation, budget, monthly forecast, unit cost, optimization.

Review and Portfolio

Produce an executive summary, context diagram, deployment diagram, data-flow diagram, ADR set, threat model, SLO, runbooks, recovery report, cost model, incident report, and backlog. Present a 20-minute review followed by challenge questions and a live troubleshooting demonstration.

Production Design Checklist

- Requirements are measurable and traced to evidence.
- Infrastructure and application are reproducibly deployed from version control.
- Security, reliability, observability, recovery, and cost are built in.
- At least one failure and one rollback or restore are tested.
- Residual risks and assumptions are honest and prioritized.
- Portfolio materials contain no secrets or confidential third-party information.

Troubleshooting Matrix

Symptom	Likely cause	Investigation	Corrective action
Capstone works only after manual fixes	Automation and configuration are incomplete	Rebuild from clean environment and record manual steps	Move every required change into IaC, pipeline, or runbook.
Recovery test misses objective	Capacity, data, or procedure assumption wrong	Compare timeline and data checkpoints	Remediate highest delay and repeat test.
Reviewers cannot follow design	Diagrams and decisions lack boundaries or outcomes	Trace one user and one failure flow	Simplify diagrams and link decisions to requirements.
Cost is unexpectedly high	Load model, logging, NAT, data, or idle resources	Analyze cost by usage and component	Fix leak, update model, add budget/anomaly control.

Hands-on Lab 70: Deliver the Final AWS Engineering Portfolio

Objective Build, validate, and defend the integrated enterprise platform.

Implementation

1. Create requirements, assumptions, risks, ADRs, architecture diagrams, and delivery plan.
2. Provision the foundation and workload using IaC and controlled pipelines.
3. Implement application, data, integration, identity, security, and observability controls.
4. Run functional, performance, security, rollback, restore, and failure tests.
5. Measure SLO, RTO, RPO, deployment time, change failure, monthly cost, and unit cost.
6. Present evidence, residual risks, and a 90-day improvement roadmap to a review panel.

Validation Evidence

- A clean environment can be reproduced from source.
- All major claims are supported by screenshots, logs, test output, metrics, or reports.
- Review panel can trace requirements to controls and outcomes.

Cleanup

- Destroy nonessential resources after export and evidence validation.
- Retain sanitized source and reports; remove credentials, account IDs, and sensitive data.

Stretch challenge Add a second tenant tier or Region and demonstrate that the platform evolves without duplicating uncontrolled architecture.

Chapter Summary

- The final capstone integrates architecture, delivery, security, operations, resilience, and FinOps.
- Professional portfolios emphasize decisions, evidence, failures, and learning - not screenshots alone.
- A reproducible, tested system is stronger proof than a large but unverified design.

Interview Questions and Model Answers

1. What is the strongest capstone evidence?

Reproducible source plus measured deployment, security, failure, recovery, and cost outcomes.

2. How should residual risk be presented?

State the condition, likelihood, impact, current controls, owner, and prioritized remediation.

3. What should happen after the capstone?

Operate a smaller version continuously, repeat failure and recovery tests, improve the platform, and use the evidence in interviews and certification preparation.

Official AWS References

AWS Well-Architected Tool: <https://docs.aws.amazon.com/wellarchitected/latest/userguide/intro.html>

AWS Architecture Center: <https://aws.amazon.com/architecture/>

AWS Resilience Hub: <https://docs.aws.amazon.com/resilience-hub/>

AWS Cost Management: <https://docs.aws.amazon.com/cost-management/>

Enterprise Project Portfolio - Ten Additional Projects

Project	Architecture focus	Required proof
1. Secure Static Web Platform	CloudFront, S3, Route 53, WAF, CI/CD	TLS, private origin, cache behavior, rollback, cost.
2. Auto-Healing Web Tier	ALB, Auto Scaling, EC2, Systems Manager	Failure replacement, patching, alarms, load result.
3. Serverless Order Workflow	API Gateway, Lambda, SQS, Step Functions, DynamoDB	Idempotency, DLQ, replay, tracing, cost per order.
4. Container Delivery Platform	ECR, ECS/Fargate or EKS, pipeline	Signed immutable image, rollout, rollback, SLO.
5. Multi-Tenant SaaS API	Identity, tenant context, pooled/silo data	Cross-tenant negative tests, quotas, tenant metrics.
6. Analytics Lakehouse	S3, Glue, Athena/Redshift, governance	Catalog, partitioning, encryption, quality, query cost.
7. Hybrid Migration Lab	VPN/TGW, Resolver, MGN/DMS plan	Dependency map, cutover, rollback, route failover.
8. Security Operations Hub	Organizations, CloudTrail, Config, GuardDuty, Security Hub	Central finding, investigation, containment, evidence.
9. Regional Recovery Platform	Replicated data, traffic control, recovery automation	Measured RTO/RPO, data reconciliation, failback.
10. Internal Developer Platform	Catalog, CDK/CloudFormation, pipeline, observability	Time to first deploy, policy, versioning, adoption feedback.

Final Interview Question Bank

1. How would you separate development, security, shared services, logging, and production accounts?
2. Design a private, highly available three-tier application and explain every route.
3. How do security groups and NACLs differ during troubleshooting?
4. When would you choose ECS, EKS, Lambda, or EC2 Auto Scaling?
5. How do you prove a deployment artifact is identical across environments?
6. Design tenant isolation for a SaaS platform with premium tenants.
7. How would you migrate a database with minimal downtime and safe rollback?
8. Explain how you would diagnose intermittent p99 latency.
9. How do you prevent retries from causing a cascading failure?
10. Design idempotency for an order-processing workflow.
11. How would you respond to a leaked access key?
12. What evidence proves backups are usable?
13. When is multi-Region justified, and what new failure modes does it introduce?
14. How do you design safe SCP rollout?
15. How would you reduce cloud cost without increasing outage risk?
16. What metrics define a platform engineering product?
17. How do you run a game day safely?

18. What makes an alert actionable?

19. How do you distinguish root cause from a downstream symptom?

20. What architecture decision did you reverse, and what evidence changed your view?

Ninety-Day AWS Engineering Practice Plan

Period	Focus	Deliverable
Days 1-15	Requirements, ADRs, architecture diagrams, account and cost safety	Two ADRs and one reviewed baseline architecture.
Days 16-30	IaC, identity, network, compute, storage	Reproducible multi-AZ sandbox foundation.
Days 31-45	Data, messaging, containers/serverless, delivery	Application deployed through immutable pipeline.
Days 46-60	Observability, security operations, patching, audit	SLO dashboard, finding workflow, runbooks.
Days 61-75	Performance, failure, restore, regional recovery	Benchmark, game day, restore and recovery report.
Days 76-90	FinOps, portfolio, certification and interviews	Final capstone review and interview evidence pack.

Architecture Review Evidence Checklist

- Requirements, constraints, assumptions, risks, and acceptance tests.
- Context, deployment, data-flow, trust-boundary, and failure-flow diagrams.
- ADRs with alternatives and consequences.
- Threat model, identity model, encryption and secret lifecycle.
- IaC validation, policy tests, artifact provenance, and deployment evidence.
- SLOs, dashboards, alarms, traces, logs, and runbooks.
- Performance, rollback, restore, failover, and fault-experiment results.
- Cost allocation, forecast, unit economics, and optimization evidence.
- Incident/postmortem evidence and remediation verification.
- Residual-risk register and 90-day improvement roadmap.

Command and Evidence Reference

Task	Example command or evidence
Confirm identity	<code>aws sts get-caller-identity</code>
List organization accounts	<code>aws organizations list-accounts</code>
CloudTrail lookup	<code>aws cloudtrail lookup-events --lookup-attributes AttributeKey=EventName,AttributeValue=EVENT</code>
Config timeline	<code>aws configservice get-resource-config-history --resource-type TYPE --resource-id ID</code>
Describe alarms	<code>aws cloudwatch describe-alarms</code>
X-Ray summaries	<code>aws xray get-trace-summaries --start-time TIME --end-time TIME</code>

Task	Example command or evidence
FIS experiments	aws fis list-experiments
Cost Explorer	aws ce get-cost-and-usage --time-period Start=YYYY-MM-DD,End=YYYY-MM-DD --granularity MONTHLY --metrics UnblendedCost
Service quotas	aws service-quotas list-service-quotas --service-code SERVICE
Architecture evidence	ADR, diagram, threat model, IaC diff, test output, SLO, recovery report, cost model

Production Readiness Graduation Checklist

- I can translate business outcomes into measurable architecture requirements.
- I can design and troubleshoot identity, network, compute, storage, data, and integration paths.
- I can deliver immutable infrastructure and application changes through controlled pipelines.
- I can define SLOs, diagnose distributed failures, and run incident command.
- I can preserve evidence, contain a cloud security event, and recover trusted service.
- I can run performance, restore, failover, and fault-injection tests safely.
- I can explain cost allocation, commitments, unit economics, and optimization risk.
- I can defend architecture trade-offs using ADRs and measured evidence.
- I can present a reproducible portfolio without exposing secrets or confidential information.
- I can identify what I do not know and create a safe validation plan.

Glossary

Term	Definition
ADR	Architectural decision record documenting context, choice, alternatives, and consequences.
Quality attribute	A measurable system property such as availability, latency, security, or recoverability.
Tenant isolation	Controls that prevent one SaaS tenant from accessing or degrading another.
Migration wave	A dependency-aware group of workloads moved through a controlled cutover.
Strangler pattern	Incremental replacement of legacy capability by routing to new components.
Split-brain	Multiple locations independently acting as authority for the same state.
Chain of custody	Evidence handling record that protects integrity and traceability.
Game day	Planned operational exercise that tests response and resilience.
Steady state	Measurable acceptable behavior used to evaluate a resilience experiment.
Unit economics	Cost associated with a meaningful business unit such as tenant, order, or request.

Term	Definition
Golden path	Supported and opinionated workflow for building and operating a common workload.
Incident commander	Role accountable for priorities, decisions, coordination, and safety during an incident.
Residual risk	Risk remaining after current controls and accepted with an owner and plan.
Evidence manifest	Index linking requirements and controls to their proof artifacts.
STAR-L	Situation, task, action, result, and learning structure for behavioral evidence.

Official Documentation Index

AWS Well-Architected Framework: <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>

AWS Architecture Center: <https://aws.amazon.com/architecture/>

AWS Prescriptive Guidance: <https://docs.aws.amazon.com/prescriptive-guidance/>

AWS Cloud Adoption Framework: <https://docs.aws.amazon.com/whitepapers/latest/overview-aws-cloud-adoption-framework/welcome.html>

AWS Security Incident Response Guide: <https://docs.aws.amazon.com/whitepapers/latest/aws-security-incident-response-guide/welcome.html>

AWS Fault Injection Service: <https://docs.aws.amazon.com/fis/>

AWS Resilience Hub: <https://docs.aws.amazon.com/resilience-hub/>

AWS Cost Management: <https://docs.aws.amazon.com/cost-management/>

AWS Certification Exam Guides: <https://docs.aws.amazon.com/aws-certification/latest/examguides/aws-certification-exam-guides.html>

AWS Architecture Decision Records: <https://docs.aws.amazon.com/prescriptive-guidance/latest/architectural-decision-records/introduction.html>

End of Volume 6F Volume 6 is now complete across Parts A-F. The next publishing step is to assemble all six parts into a complete master DOCX and PDF with unified navigation, glossary, and index.